

Clasificador de Señales en Tiempo Real sobre STM32 utilizando FreeRTOS y NanoEdge AI

Matías Oliva

29 de junio de 2026

Índice

1. Introducción	2
2. Objetivos	2
3. Hardware Utilizado	2
4. Arquitectura General	3
5. Interface de usuario (tarea StartUartTask)	3
6. Generación de Señales (tarea StartDacTask)	3
6.1. Control del DAC	4
7. Adquisición de Datos (tarea StartAdcTask)	4
7.1. Half Transfer y Transfer Complete	4
8. Dataset para Entrenamiento	5
9. Entrenamiento con NanoEdge AI Studio	5
10. Implementación del Modelo en STM32	6
11. Migración a FreeRTOS	7
12. Aplicación de Monitoreo en Python	7
13. Resultados	7
14. Conclusiones	8
15. Trabajo Futuro	8

1. Introducción

Este informe describe el desarrollo de un sistema embebido capaz de generar, adquirir y clasificar señales en tiempo real utilizando una placa STM32 NUCLEO-U575ZI-Q.

El proyecto combina técnicas de procesamiento digital de señales, sistemas operativos de tiempo real y aprendizaje automático embebido mediante NanoEdge AI Studio.

El objetivo principal es identificar automáticamente diferentes formas de onda generadas por el propio sistema utilizando un modelo de clasificación ejecutado localmente en el microcontrolador.

El proyecto está disponible en https://github.com/ushikawa93/signal_classifier_with_rtos_stm32.

2. Objetivos

- Generar señales sinusoidales, cuadradas y triangulares utilizando el DAC.
- Adquirir las señales mediante el ADC.
- Construir un dataset para entrenamiento.
- Entrenar un clasificador utilizando NanoEdge AI Studio.
- Implementar el modelo en un STM32.
- Utilizar FreeRTOS para coordinar las distintas tareas del sistema.
- Desarrollar una interfaz gráfica para monitoreo y captura de datos.

3. Hardware Utilizado

- STM32 NUCLEO-U575ZI-Q
- DAC interno
- ADC interno
- Comunicación UART
- Cable de conexión DAC-ADC

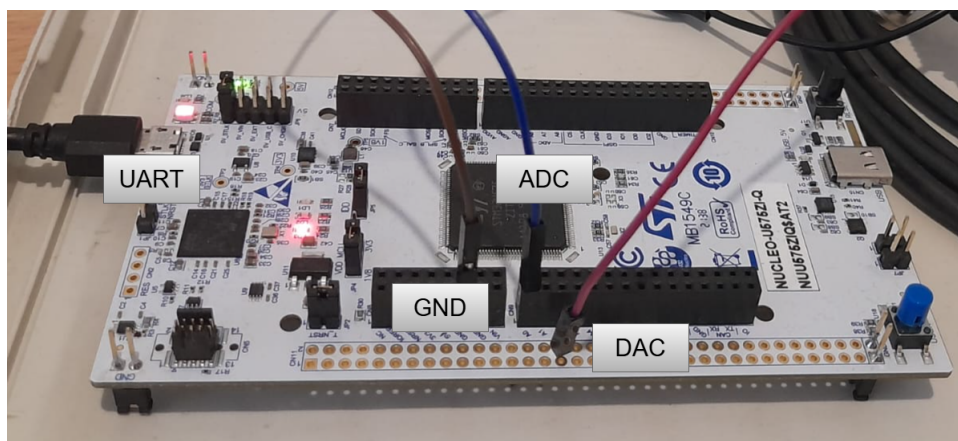


Figura 1: Montaje experimental

4. Arquitectura General

La arquitectura implementada se basa en la generación de señales mediante DAC, adquisición mediante ADC y posterior clasificación utilizando un modelo de Machine Learning.

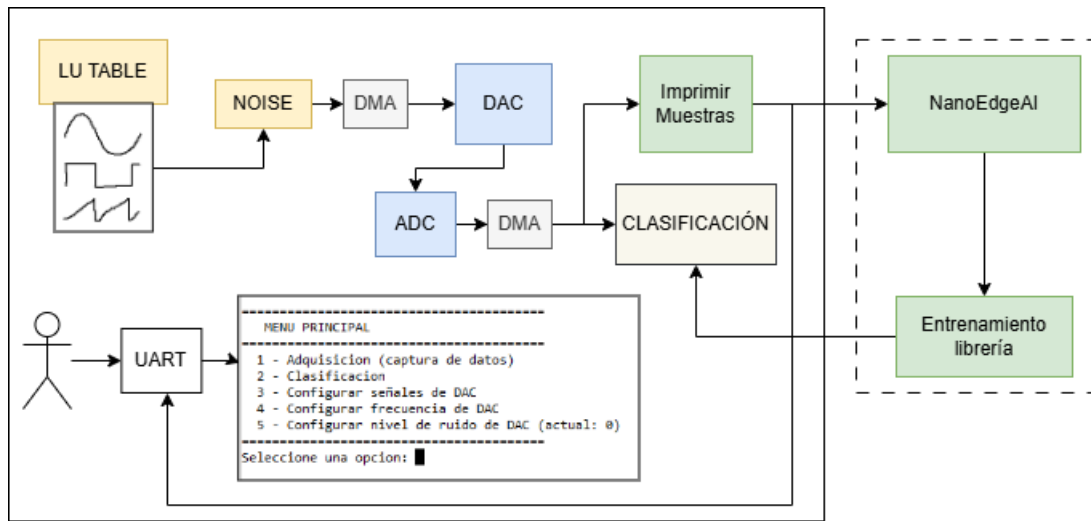


Figura 2: Arquitectura general

5. Interface de usuario (tarea StartUartTask)

A través del puerto UART el sistema presenta al usuario un menú definido en el header `menu.h`. En este se le presentan las opciones:

- 1 - Adquisicion (captura de datos)
- 2 - Clasificacion
- 3 - Configurar señales de DAC
- 4 - Configurar frecuencia de DAC
- 5 - Configurar nivel de ruido de DAC

La operación del menú y su derivación a las tareas correspondientes están controladas por la tarea `StartUartTask`.

6. Generación de Señales (tarea StartDacTask)

La generación de señales se implementó utilizando:

- DAC interno (12 bits).
- DMA en modo circular.
- Temporizador como trigger.
- Tablas precalculadas de muestras.
- Generador de ruido interno del STM32.

Las formas de onda implementadas son:

- Sinusoidal.
- Cuadrada.
- Triangular.

6.1. Control del DAC

La tarea `StartDacTask` (definido en `app_freertos.h`) junto con el header `signals.h` controlan la operación del DAC.

Esta tarea comienza agregándole ruido a las LU tables (`generateSignalsWithNoise(int)`) y luego lanza el DAC por DMA. Este esquema tiene la limitación de que el ruido se calcula solo en este punto, por lo que todos los ciclos de la señal tienen los mismos esquemas de ruido (es decir, el ruido es periódico). En un futuro podría mejorarse este punto recalculando las tablas en alguna interrupción que se de al finalizar el buffer del DMA.

La transferencia memoria- DAC es disparada por el temporizador TIM2. Por defecto este timer está configurado para generar una señal de 1 segundo de periodo, a partir de una LU table de 128 puntos (triggers cada 1/128 s).

Esta tarea está atenta a tres eventos que pueden dispararse desde la UART que permiten modificar la forma de onda:

- `DAC_WAVE`: Cambio de la forma de onda.
- `DAC_NOISE`: Cambia en nivel de ruido en la señal. Para ello detiene el DAC y vuelve a llamar a `generateSignalsWithNoise(int)`. El nivel de ruido se puede configurar de 0 a 4095 (cuentas del DAC).
- `DAC_FREQ`: Cambia la frecuencia de actualización del DAC, llamando a la función `getTIMER_DAC(int)` para calcular el periodo del timer deseado a partir de la frecuencia de señal objetivo.

7. Adquisición de Datos (tarea `StartAdcTask`)

La adquisición se realiza mediante el ADC utilizando DMA circular. Su ejecución está definida en la tarea `StartAdcTask`. Esta configura la adquisición por DMA al principio de la operación y luego queda esperando eventos que señalizan bloques de datos listos para procesar.

Para ello se implementó un buffer doble de 256 muestras para permitir el procesamiento simultáneo de los datos mientras continúa la adquisición. La transacción del DMA es disparada por el TIM15, configurado para funcionar a 100 Hz (frecuencia de muestreo).

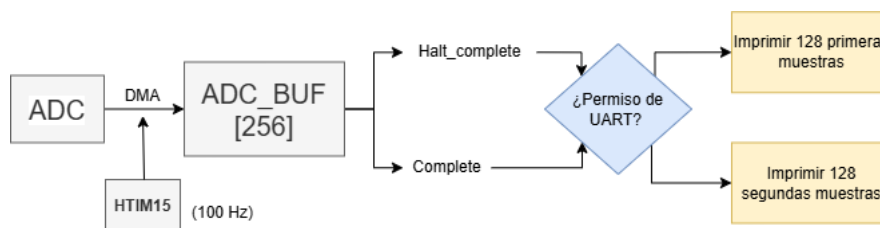


Figura 3: Configuración ADC-DMA

7.1. Half Transfer y Transfer Complete

A medida que se llena el buffer del ADC se disparan dos interrupciones que le avisan a la tarea `StartAdcTask` que hay datos disponibles para procesar

- **Half Complete.** Las primeras 128 muestras del buffer están disponibles
- **Complete.** Las segundas 128 muestras del buffer están disponibles.

En esta etapa del desarrollo de este proyecto el procesamiento de los datos consiste en imprimir, si el usuario pidió datos desde la UART, los bloques correspondientes a través del puerto serie.

8. Dataset para Entrenamiento

Para entrenar el clasificador se implementó un modo de captura que transmite muestras por UART en formato JSON.

Ejemplo:

```
{
  "ts": "00:00:00",
  "Muestras": [...]
}
```

Cuando el usuario lo pide se imprimen por puerto UART muestras en este formato. Estos se pueden almacenar y utilizar para entrenar algoritmos de clasificación utilizando NanoEdge AI Studio.

```
=====
MENU PRINCIPAL
=====
1 - Adquisicion (captura de datos)
2 - Clasificacion
3 - Configurar señales de DAC
4 - Configurar frecuencia de DAC
5 - Configurar nivel de ruido de DAC (actual: 0)
=====
Seleccione una opcion: 1
Ingrese numero de datos a adquirir ('c' para adquisicion continua)...
1
{ "ts":00:00:00 , "Muestras": 6483, 6854, 7289, 7621, 8034, 8414, 8864, 9218, 962
5, 10008, 10404, 10728, 11159, 11498, 11917, 12241, 12532, 12889, 13150, 13500, 13
783, 14047, 14398, 14564, 14806, 15014, 15221, 15432, 15584, 15679, 15840, 15970,
16020, 16116, 16142, 16175, 16217, 16211, 16169, 16113, 16021, 15960, 15835, 15750
, 15609, 15429, 15237, 14923, 14837, 14716, 14378, 14109, 13830, 13520, 13092, 129
38, 12609, 12281, 11936, 11564, 11186, 10840, 10462, 10075, 9695, 9282, 8894, 8509
, 8104, 7733, 7299, 6921, 6524, 6148, 5784, 5379, 5023, 4654, 4335, 3964, 3646, 32
54, 2985, 2702, 2401, 2145, 1882, 1644, 1406, 1223, 977, 775, 645, 491, 380, 248,
215, 168, 40, 42, 131, 44, 76, 137, 168, 277, 344, 490, 674, 782, 977, 1110, 1298,
1684, 1839, 2126, 2355, 2555, 2861, 3270, 3646, 3888, 4152, 4590, 4954, 5277, 579
0, 6063, }
```

Figura 4: Captura de datos para entrenamiento

9. Entrenamiento con NanoEdge AI Studio

Utilizando la plataforma se adquirieron señales de distintas frecuencias, sinusoidales, ondas cuadradas y triangulares que fueron utilizadas para entrenar un modelo de clasificación en NanoEdge AI Studio. Para adaptar las señales al formato csv requerido por el software se utilizaron scripts disponibles en la carpeta `/utils` del proyecto. En esta carpeta también hay disponible un script de matlab para graficar segmentos de las señales obtenidas y verificar su integridad.

- **Dataset utilizado:** Los datos específicos que se utilizaron para entrenar el modelo se encuentran en la carpeta `nanoEdgeAI/dat`.

- **Número de clases:** El modelo se configuró con tres clases: *OndaSinusoidal*, *OndaCuadrada* y *OndaTriangular*.
- **Configuración del entrenamiento:** Se empleó la versión 5.1.1 de NanoEdge AI Studio sobre la plataforma NUCLEO-U575ZI-Q, con señales de 128 columnas por muestra y 9 señales en total por clase. La búsqueda de la librería óptima se realizó mediante el benchmark automático (*autoML*) de la herramienta, evaluando 17838 librerías candidatas durante 42 minutos con la configuración por defecto.
- **Resultados obtenidos:** El benchmark finalizó seleccionando una librería con un *Quality Index* de 99/100 puntos y una *Balanced Accuracy* del 100 %, con un consumo de memoria de 2.4 KB de RAM (incluyendo buffer de 0.6 KB) y 2.7 KB de Flash. La matriz de confusión sobre el conjunto de validación no presentó ningún error de clasificación entre las tres clases, tal como se muestra en la Figura 5.

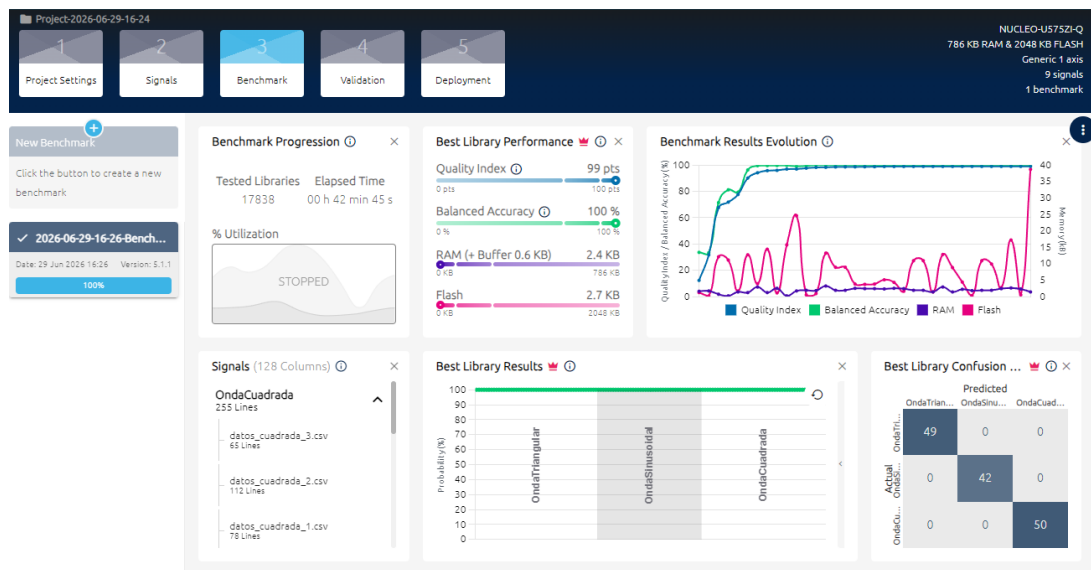


Figura 5: Resultados finales del benchmark en NanoEdge AI Studio.

10. Implementación del Modelo en STM32

El modelo generado fue integrado al firmware mediante las bibliotecas proporcionadas por NanoEdge AI Studio.

El modelo se inicializa antes de lanzar las tareas, mediante la función `neai_classification_init()` provista en la librería.

Luego, dentro de la tarea que controla el UART en la librería, se agregó la clasificación de señales. En este punto, cuando es requerido por el usuario se utiliza la función `neai_classification()`, pasándole el contenido del buffer proveniente del ADC que este completo en ese momento (puede ser la primera o la segunda mitad del `ADC_BUF`). Antes de utilizarse directamente las muestras se convierten de entero a punto flotante, por requerimientos de la librería.

Usando este esquema el sistema decide entre alguna de las tres ondas lo que está muestreando en el ADC, y expone los resultados junto con el nivel de similitud obtenido.

Captura de pantalla de clasificador

Figura 6: Proceso de clasificación

11. Migración a FreeRTOS

Inicialmente el sistema fue desarrollado utilizando una arquitectura secuencial, con modods configurados a través de `defines`. Esta implementación preeliminar está disponible en https://github.com/ushikawa93/signal_classifier_stm32.

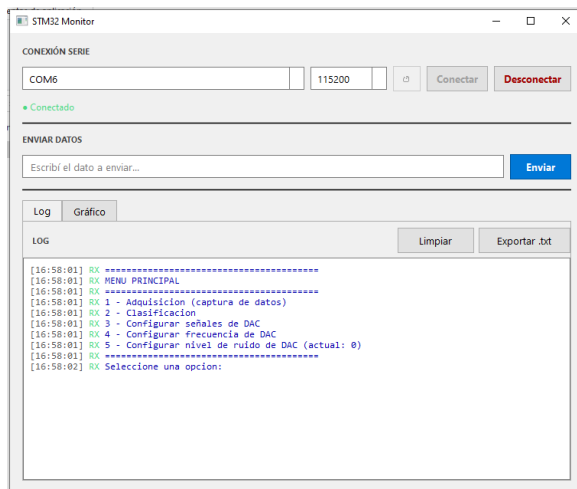
Posteriormente se migró a FreeRTOS para mejorar la organización del software y desacoplar las distintas funcionalidades.

12. Aplicación de Monitoreo en Python

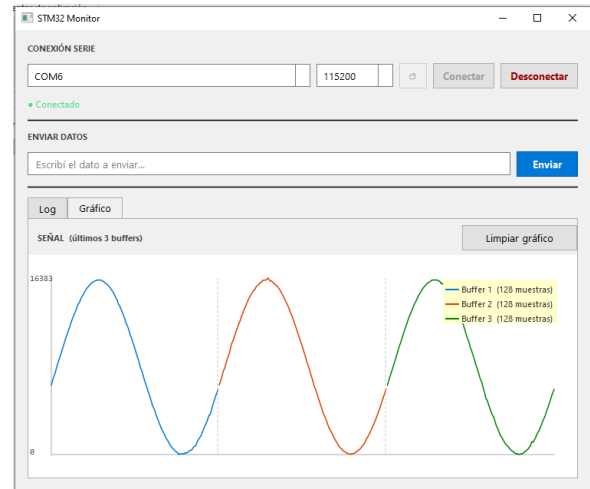
Se desarrolló una aplicación de escritorio utilizando PySide6 para interactuar con el sistema. Las funcionalidades implementadas incluyen:

- Conexión serie.
- Visualización de mensajes UART.
- Captura de buffers.
- Graficación en tiempo real.
- Exportación de logs.

Esta implementación está disponible en `/python_gui`, con un ejecutable ubicado en `/python_gui/dist/main.exe`



(a) Descripción imagen 1



(b) Descripción imagen 2

Figura 7: Aplicación de Monitoreo en Python

13. Resultados

Presentar ejemplos de:

- Señales adquiridas.
- Clasificaciones obtenidas.
- Nivel de confianza.

- Desempeño del sistema.

Insertar resultados de clasificación

Figura 8: Resultados obtenidos

14. Conclusiones

El proyecto permitió integrar distintas tecnologías dentro de una única plataforma embebida:

- STM32.
- DMA.
- FreeRTOS.
- NanoEdge AI.
- Comunicación UART.
- Aplicaciones de escritorio en Python.

Además, se obtuvo un sistema capaz de clasificar señales en tiempo real utilizando recursos limitados de procesamiento.

15. Trabajo Futuro

- Incorporar señales externas.
- Agregar nuevas clases.
- Implementar FFT en tiempo real.
- Incorporar almacenamiento en memoria SD.
- Visualización remota mediante red.